SharePoint Search Connector — визуальный план

> Визуальная версия плана из `plan.sharepoint.md`. В ней основной сценарий показан как UI-прототипы и диаграммы; короткие пояснения фиксируют контракт и оставшиеся работы.

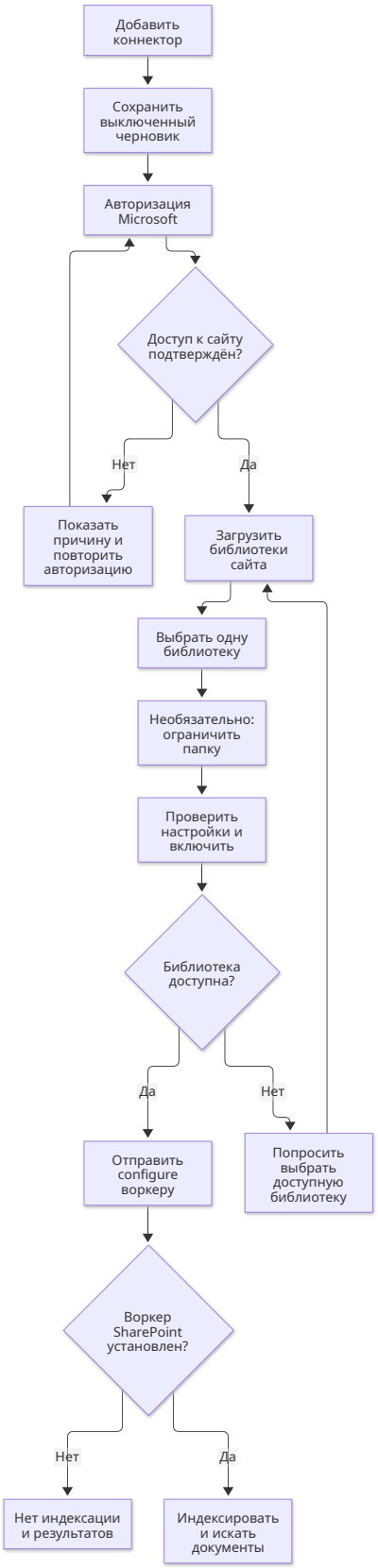
Состояние по исходному плану

Сервер и OAuth	Web UI	Rust-воркер SharePoint	Проверка с Microsoft tenant
✅ Реализованы по сводке и разделу B	✅ Реализован по сводке и разделу C	⏳ Отдельная реализация ещё нужна	⏳ Live-проверка не выполнялась

Пока нет воркера, полноценная индексация и поиск документов SharePoint не заработают. Статус `ONLINE` в прототипе — целевое состояние полного сценария после подключения воркера.

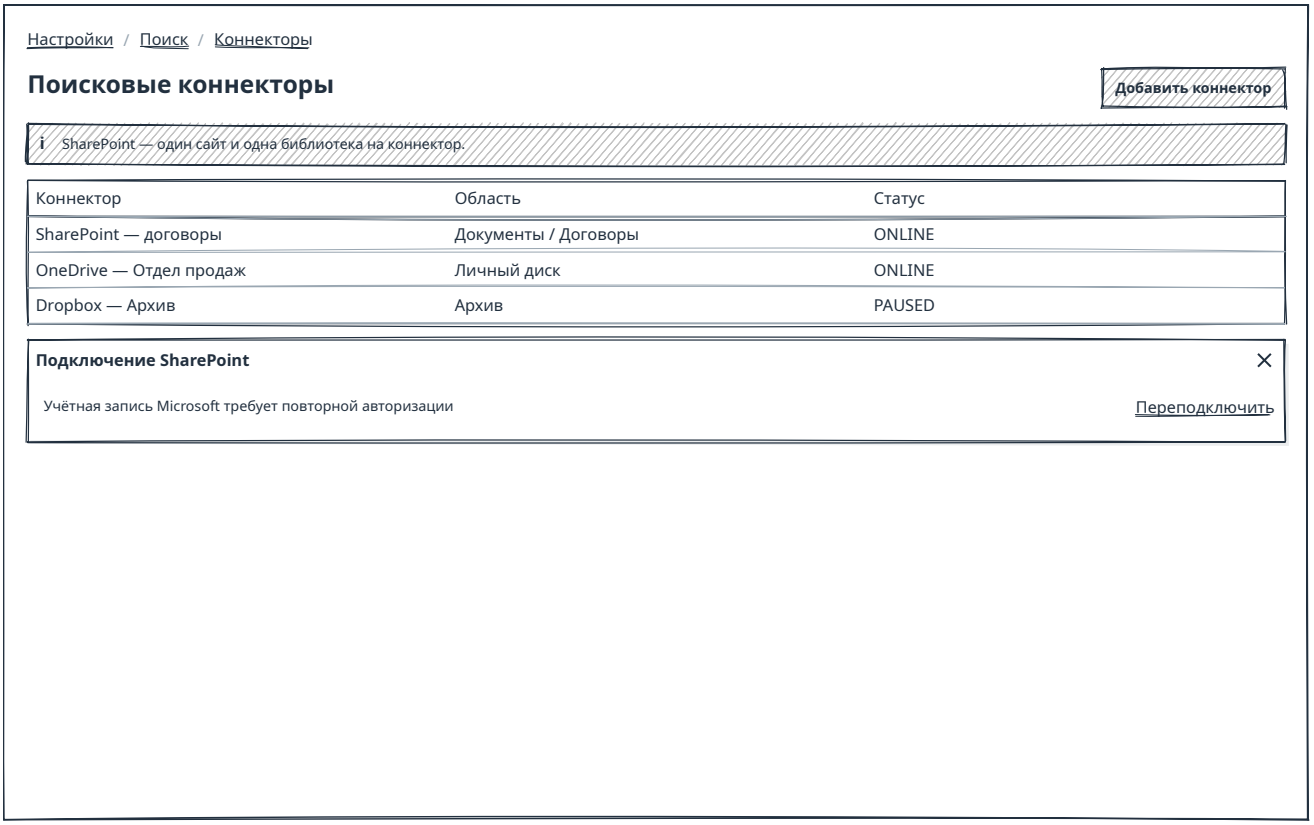
> **Сверка статуса.** В сводке и пунктах B.5–B.8 сказано, что OAuth, Graph-доступ и проверка выбранной библиотеки реализованы. В пунктах A.1–A.2 остались формулировки о незавершённой Graph-проверке. Здесь за основу взят более поздний статус из сводки и раздела B; перед следующим релизом стоит сверить эти два пункта с кодом.

Путь администратора

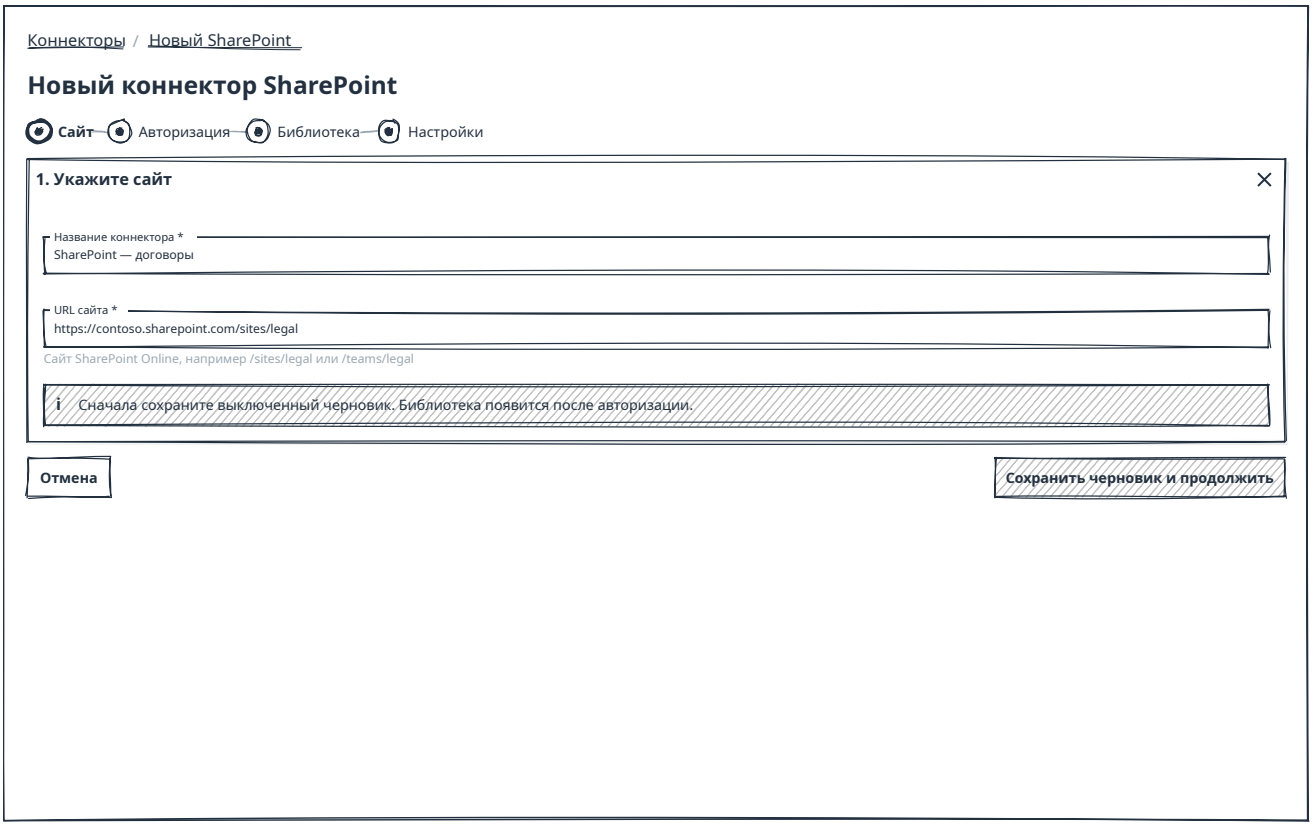


UI-прототипы

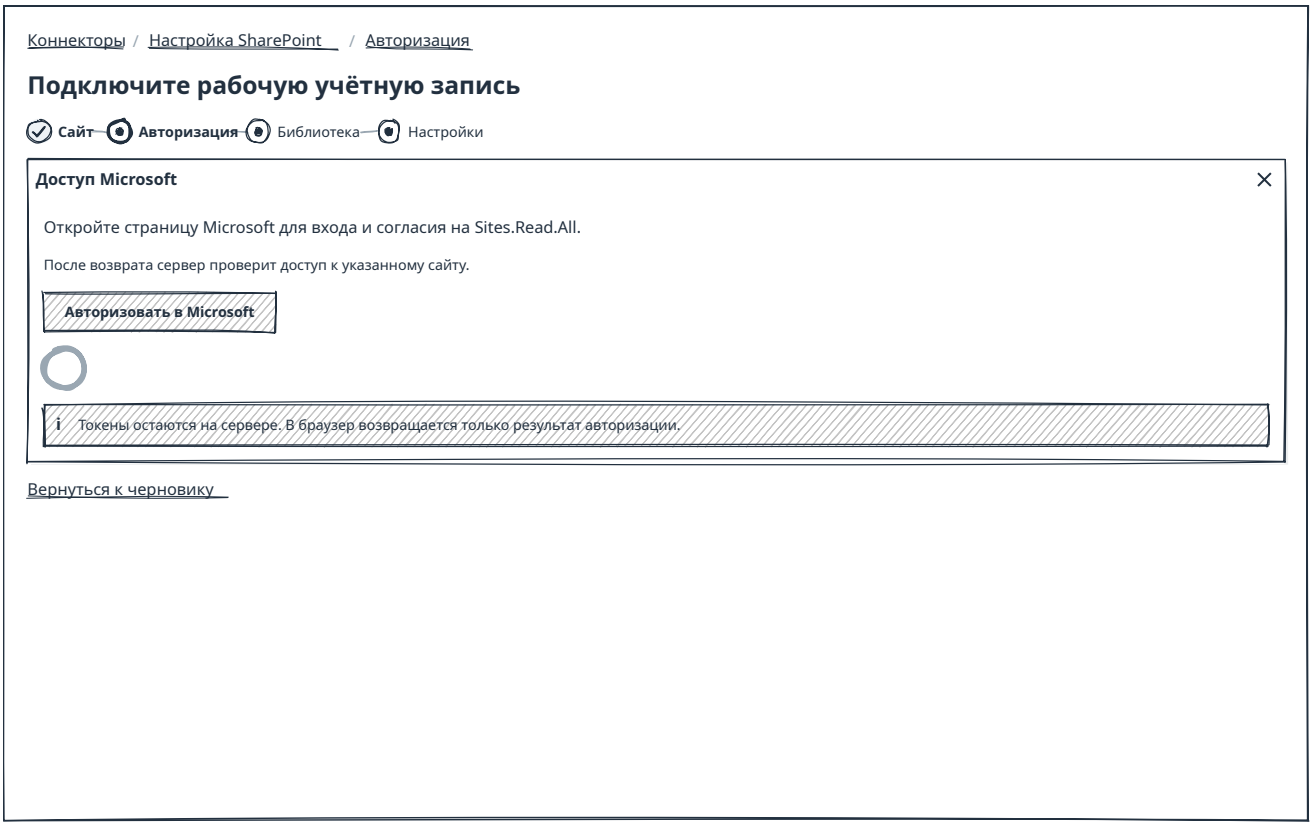
Ниже каждый экран показан отдельной диаграммой Wiremark, поэтому его можно читать и рендерить независимо. Формат: Wiremark — инструкция для LLM.



Черновик коннектора



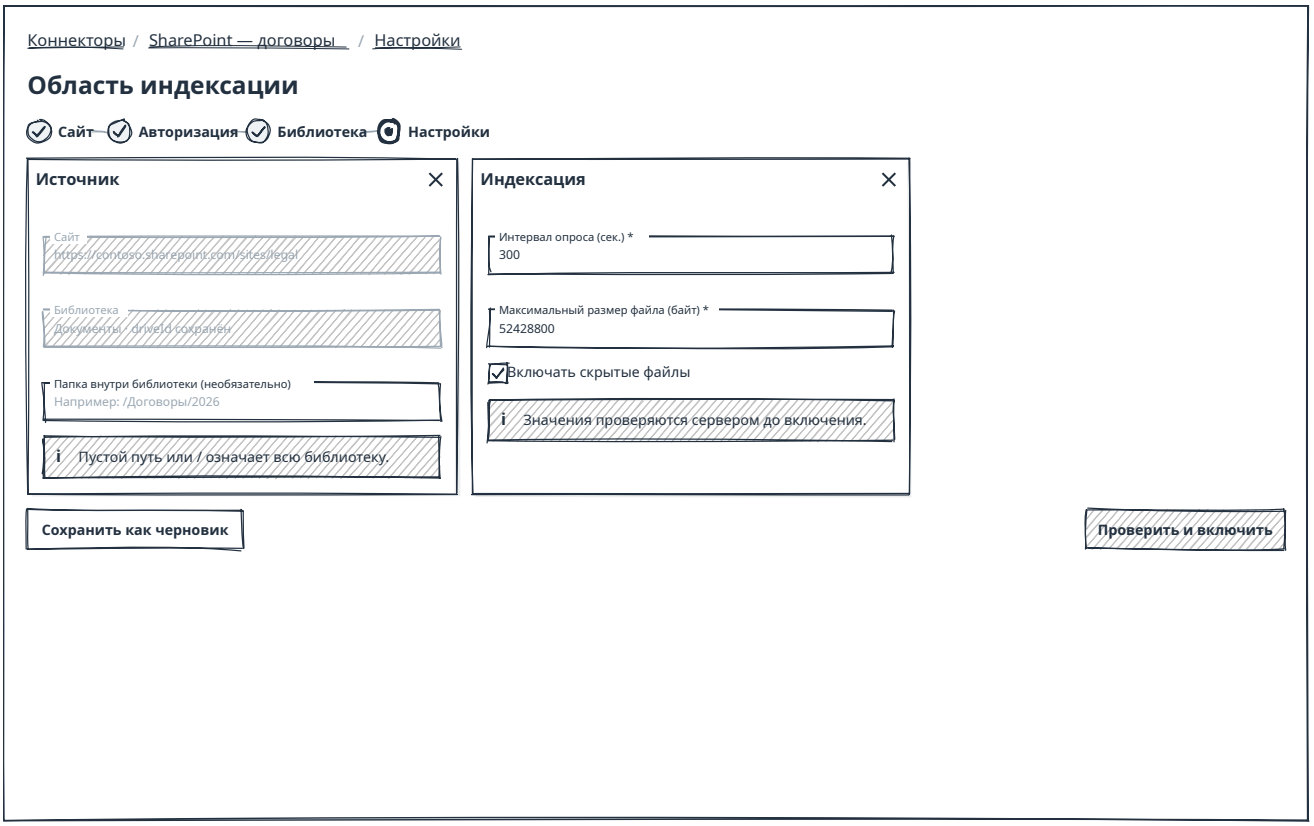
Авторизация Microsoft



Выбор библиотеки



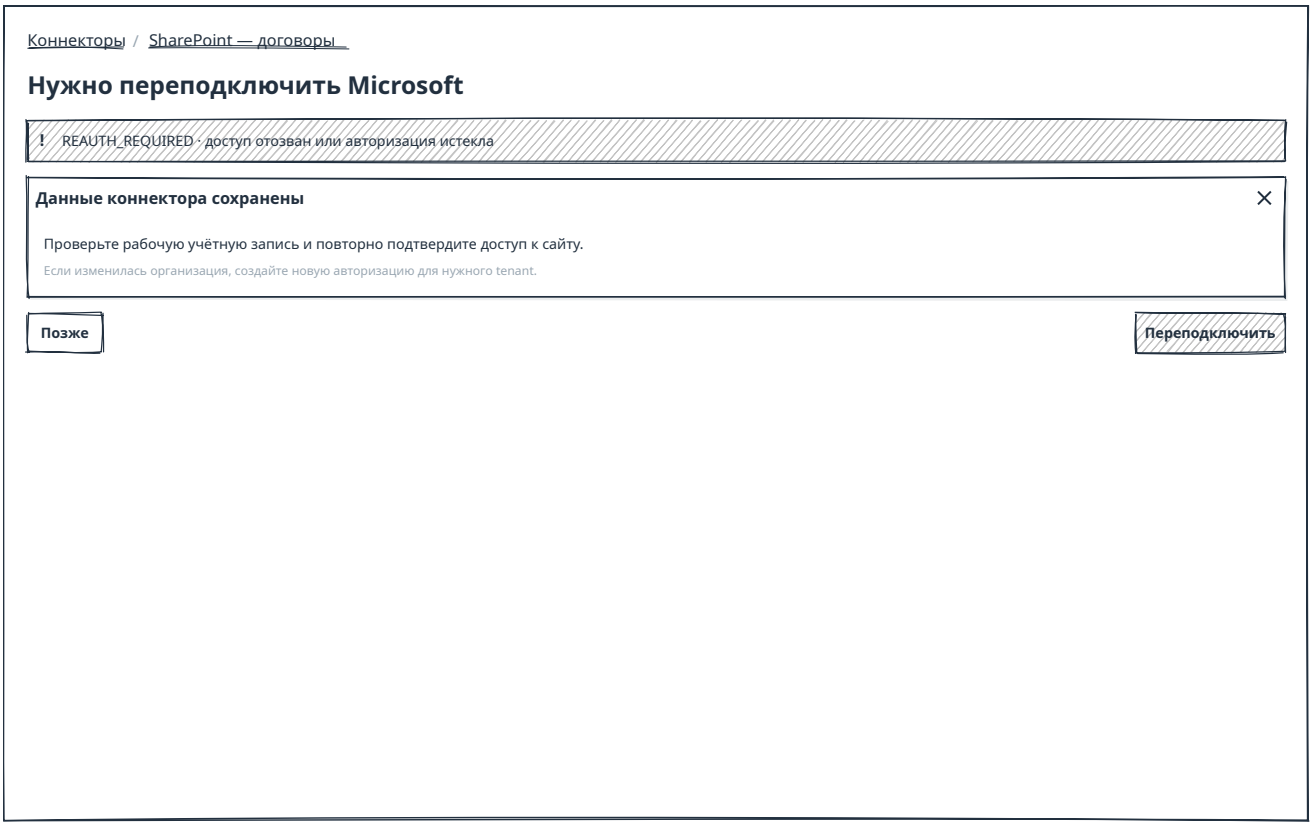
Настройки индексации



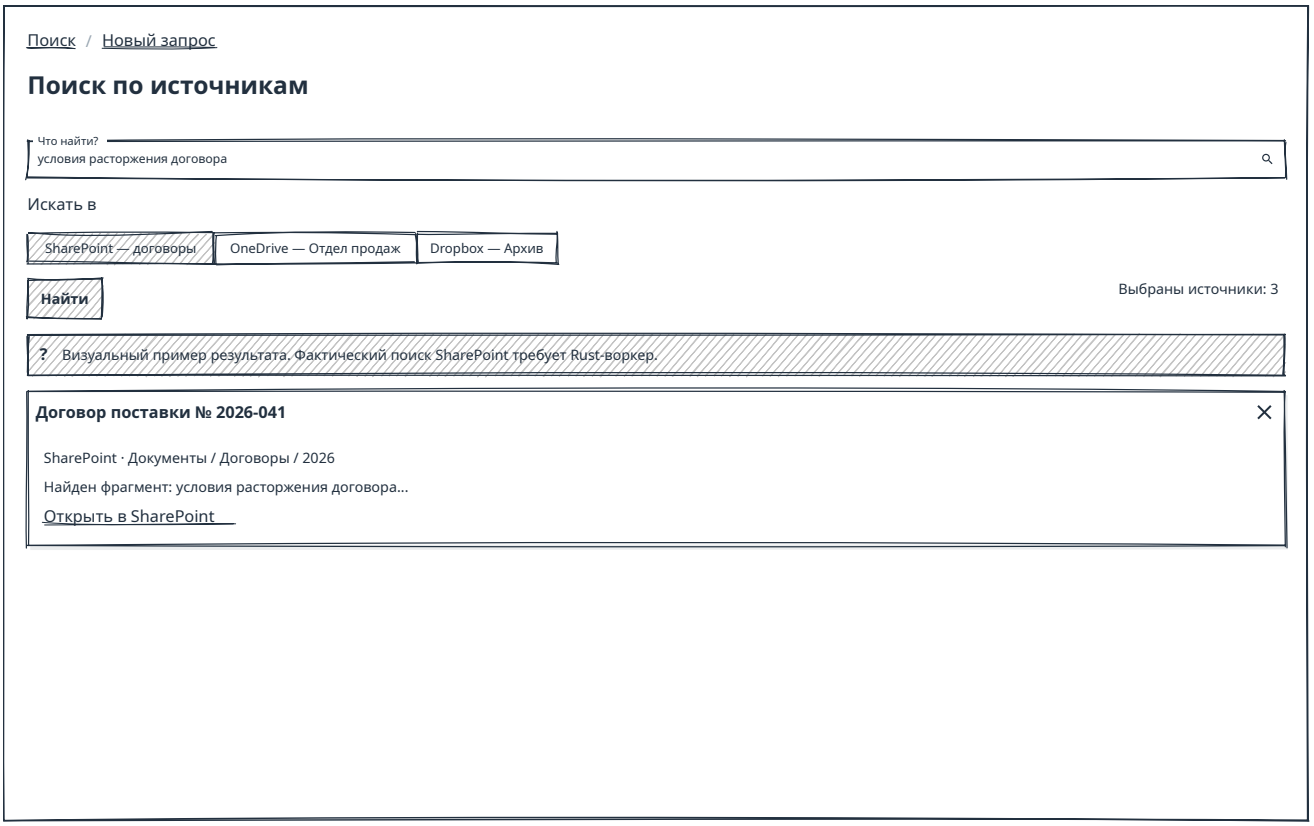
Подключённый источник



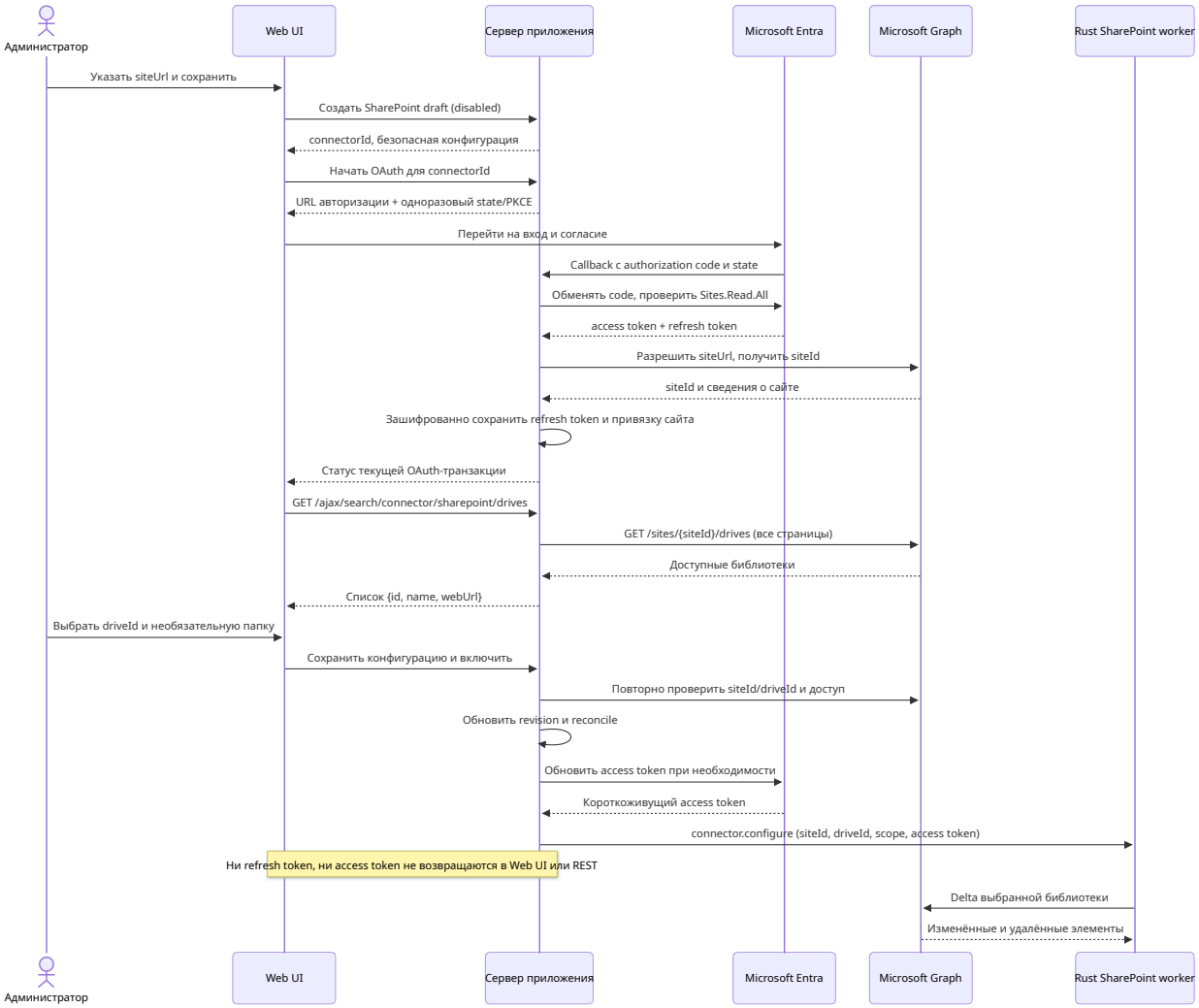
Повторная авторизация

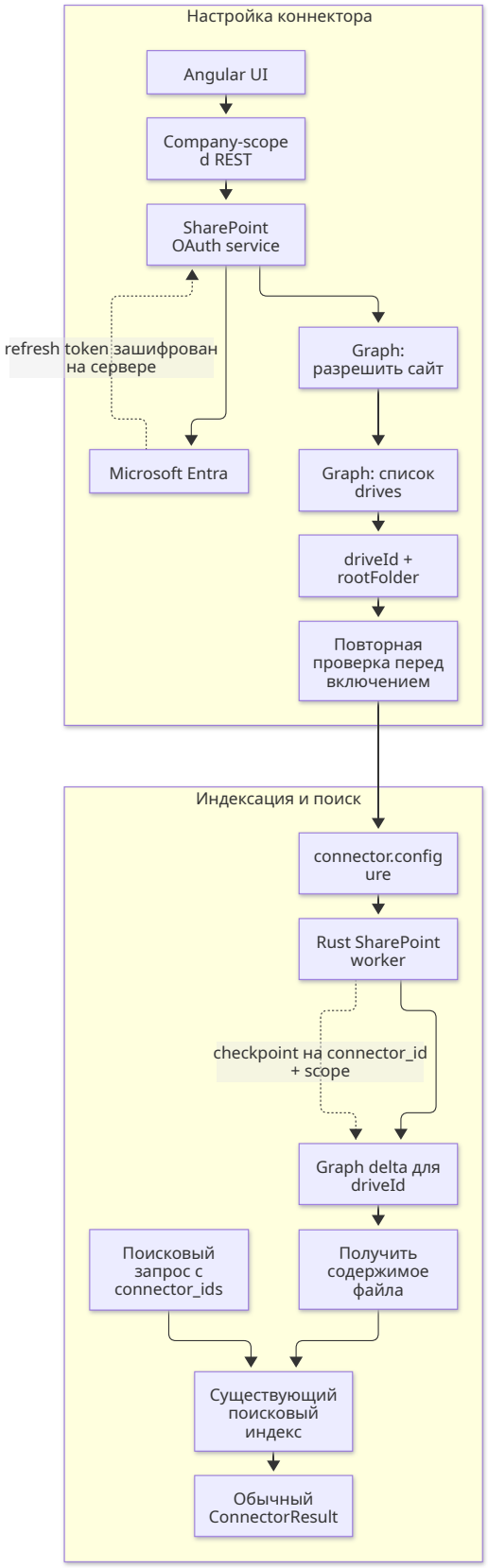


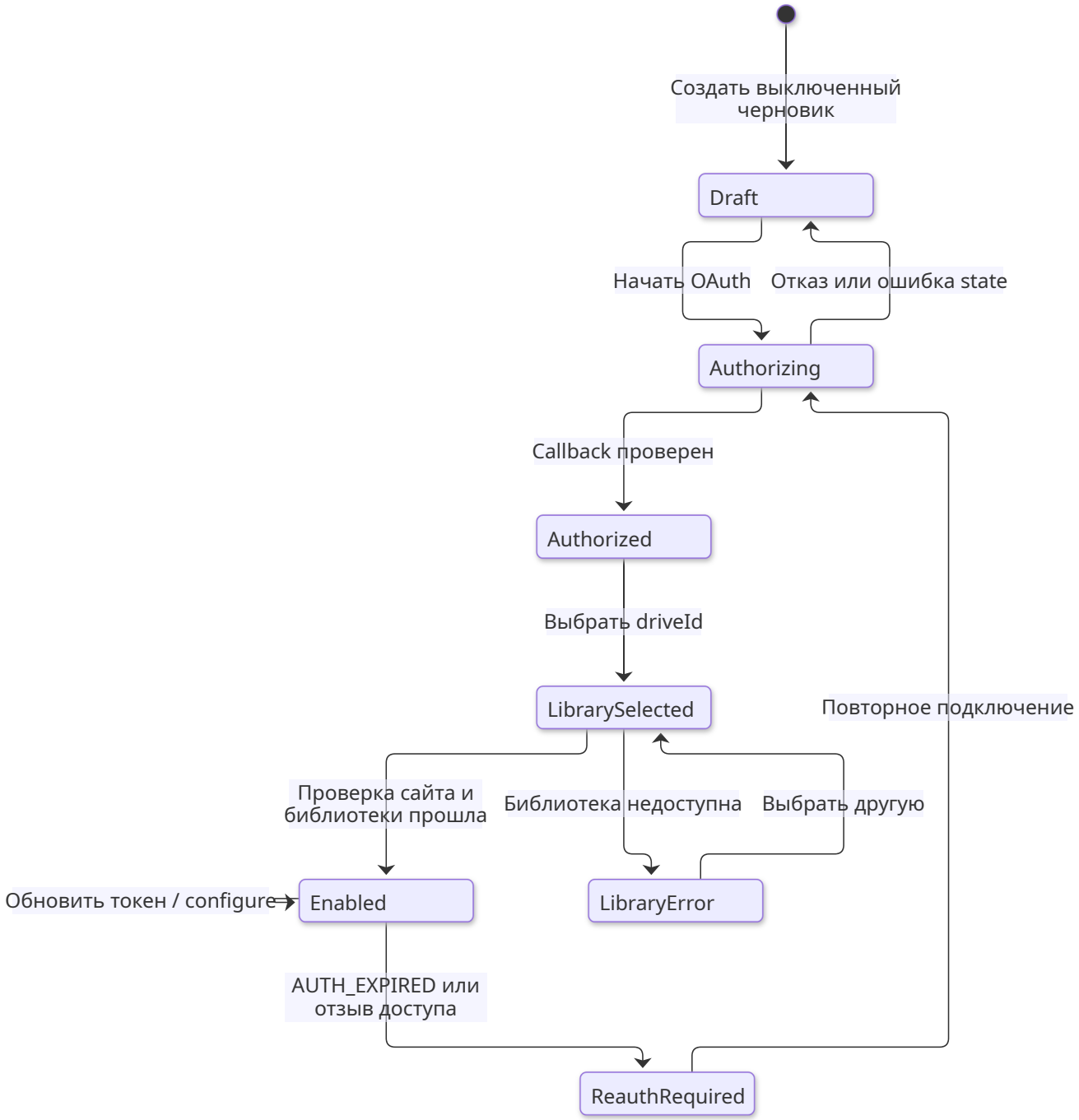
Поиск по источникам



Обмен данными и границы безопасности




Настройка коннектора
Angular UI
Company-scoped REST
SharePoint OAuth service
refresh token зашифрован на сервере
Microsoft Entra
Graph: разрешить сайт
Graph: список drives
driveId + rootFolder
Повторная проверка перед включением
Индексация и поиск
connector.configure
Rust SharePoint worker
checkpoint на connector_id + scope
Graph delta для driveId
Поисковый запрос с connector_ids
Получить содержимое файла
Существующий поисковый индекс
Обычный ConnectorResult



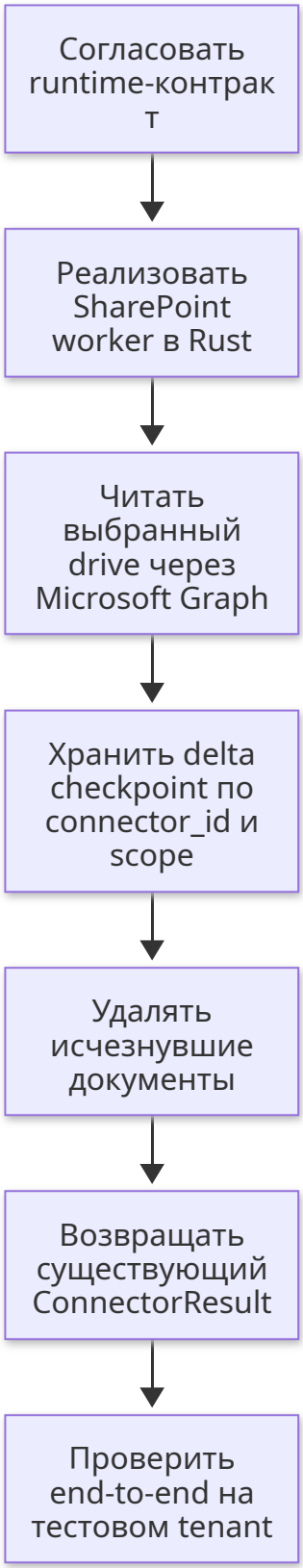
Контракт одного коннектора

Поле	Что означает
`type`	`sharepoint` — отдельный тип, OneDrive остаётся отдельным коннектором.
`siteUrl`	Канонический HTTPS URL одного сайта SharePoint Online.

Поле	Что означает
`driveId`	ID одной выбранной библиотеки; пуст до выбора.
`rootFolder`	Путь или DriveItem ID внутри библиотеки; пусто или `/` — вся библиотека.
`includeHidden`, `maxFileBytes`, `pollIntervalSeconds`	Настройки индексации с валидацией Java и Angular.
OAuth credentials	Зашифрованные на сервере refresh token, siteId, tenantId и scope. REST возвращает только признак наличия credentials и безопасный статус.
Runtime credentials	Короткоживущий access token, siteId, driveId и scope только в `connector.configure`; refresh token воркеру не передаётся.

При смене сайта библиотека и статус проверки сбрасываются. Новый сайт требует новой OAuth-привязки; смена tenant требует новой авторизации. Библиотека и папка входят в revision scope, чтобы воркер не продолжал индексировать прежнюю область.

Оставшаяся работа



- Реализовать сканер в Rust-модуле поиска по согласованному runtime-контракту. Формат `ConnectorResult` и механизм выбора цели поиска на сервере не менять.

- Вести отдельный delta checkpoint для каждого `connector_id` и scope; обрабатывать изменения и удаления файлов.
- Различать безопасные состояния: `REAUTH_REQUIRED`, отсутствие доступа, удалённая библиотека, 429 с `Retry-After` и временные ошибки Graph.
- В настройках развёртывания нужны зарегистрированное приложение Microsoft Entra, tenant, callback URL и admin consent для `Sites.Read.All`. Первый вариант OAuth — delegated с `offline_access`.
- После реализации воркера пройти ручной сценарий: OAuth → сайт и библиотека → `ONLINE` → выбрать источник в поиске → получить SharePoint-файл; повторить после истечения токена и отзыва доступа.

Проверки, записанные в исходном плане

Проверка	Статус в исходнике
`mvn -q -DskipTests compile` в модуле сервера	✅ Успешно
`mvn -q test` в модуле сервера	✅ Успешно
`npm run build -- --configuration development` в модуле Web UI	✅ Успешно
Live-интеграция с реальным Microsoft tenant	⏳ Не выполнялась
Сквозной поиск SharePoint-файла	⏳ После реализации Rust-воркера

Критерий безопасности для готового сценария: токены не попадают в REST-ответы, браузер или логи; секреты не хранятся в `configJson`. Существующие OneDrive, Dropbox и IMAP продолжают работать через общий механизм коннекторов.

Ссылки Graph API

- Получение сайта по hostname и относительному пути
- Список библиотек сайта
- Delta для файлов и скачивание содержимого
- Разрешения Graph и 429 / throttling